

Fundamentos de Inteligencia Artificial
Clásica:
Técnicas, Métodos y Aplicaciones
Antes de la IA Generativa

Luis Chamba-Eras

Editor

2026

Índice general

I	Introducción y Fundamentos Metodológicos	1
II	Introducción al Procesamiento del Lenguaje y Modelos Generativos Base	2
1.	Optimización de la Representación Semántica	3
1.1.	Introducción	3
1.2.	Fundamentos Teóricos	4
1.2.1.	Formalización del Espacio Vectorial y Métricas de Proximidad	4
1.2.2.	Modelos Estadísticos Predictivos: La Arquitectura Word2Vec	5
1.2.3.	Modelos de Factorización Matricial: Global Vectors (GloVe)	6
1.2.4.	Modelo de Subpalabras: FastText	6
1.2.5.	Modelos Dinámicos y Contextuales: La Arquitectura Transformer y BERT	7
1.2.6.	Geometría Semántica Avanzada e Isotropía del Hiperespacio	8
1.3.	Metodología y Análisis de la Literatura	9
1.3.1.	Metodología de Jerarquización y Selección Científica	9
1.3.2.	Clasificación Detallada de la Literatura Científica	10
1.3.3.	Análisis de Rendimiento Computacional y Compromisos de Infraestructura	10
1.3.4.	Evaluación Basada en Benchmarks Estandarizados	11
1.3.5.	Ingeniería de Sondas (<i>Probing Structural</i>): Auditoría de Capas Ocultas	12
1.3.6.	Destilación de Conocimiento Contextual: El Algoritmo X2Static	12
1.4.	Conclusiones	13

Parte I

Introducción y Fundamentos Metodológicos

Parte II

Introducción al Procesamiento del Lenguaje y Modelos Generativos Base

Capítulo 1

Optimización de la Representación Semántica del Lenguaje mediante el Uso de Embeddings

Autor: Yandri Alexander Piscocama Jaramillo (yandri.piscocama@unl.edu.ec)
Técnica asignada: Embeddings

1.1. Introducción

El Procesamiento del Lenguaje Natural (NLP) ha experimentado una evolución paradigmática en la ingeniería de software moderna, impulsado por la necesidad crítica de dotar a los sistemas computacionales de la capacidad de comprender, interpretar y manipular la semántica del lenguaje humano [1, 2]. A diferencia de los datos estructurados o numéricos que las máquinas procesan de manera nativa, el lenguaje natural se caracteriza por ser intrínsecamente ambiguo, discreto, altamente dependiente del contexto y propenso a variaciones morfológicas complejas. Históricamente, la informática abordó esta problemática mediante el diseño de sistemas basados en reglas sintácticas rígidas y diccionarios expertos; sin embargo, la incapacidad de estas aproximaciones para generalizar ante la evolución lingüística y el volumen masivo de datos no estructurados precipitó la adopción de modelos estadísticos y probabilísticos [3].

En las etapas iniciales de esta transición digital, los esfuerzos se concentraron en representaciones simbólicas basadas en frecuencias léxicas. Técnicas tradicionales como One-Hot Encoding (OHE), Bag of Words (BoW) y el Modelo de Espacio Vectorial basado en TF-IDF se convirtieron en el estándar de la industria. A pesar de su simplicidad de implementación, estas metodologías imponen restricciones estructurales severas que impiden una verdadera comprensión semántica profunda [4]. El primer obstáculo crítico es la denominada “maldición de la dimensionalidad”. En esquemas como OHE o BoW, cada palabra única dentro de un corpus se proyecta como una dimensión ortogonal independiente en un espacio vectorial discretizado. Consecuentemente, el tamaño de cada vector es equivalente a la cardinalidad total del vocabulario ($|V|$), derivando en matrices con una dispersión extrema (*sparsity*).

Desde una perspectiva matemática y geométrica, el fallo definitivo de los modelos tradicionales radica en la ortogonalidad de sus vectores. Al tratar cada unidad léxica de manera aislada, el producto punto entre dos vectores correspondientes a palabras conceptualmente sinónimas resulta invariablemente en cero. Como señalan Pilehvar y Camacho-Collados [5], esta configuración carece por completo de una métrica de distancia continua que mapee la similitud del mundo real en el hiperespacio, obligando a los algoritmos a depender de la coincidencia exacta de caracteres superficiales (*surface-level lexical overlap*).

Para superar este abismo técnico, la ingeniería de IA adoptó un quiebre filosófico fundamentado en la **Hipótesis Distribucional**, un principio lingüístico que postula que las palabras que ocurren en contextos similares tienden a codificar significados similares [6]. Este cambio conceptual permitió desplazar el enfoque desde los índices simbólicos discretos hacia la construcción de espacios vectoriales continuos y densos de baja dimensionalidad ($\mathbb{R}^d$). En lugar de mapear palabras en dimensiones masivas proporcionales a $|V|$, los modelos de *embeddings* comprimen la esencia semántica del vocabulario en vectores de tamaño fijo y denso, típicamente parametrizados entre 100 y 768 dimensiones [5].

La principal ventaja de este espacio continuo es la emergencia de una geometría semántica interpretable. Al proyectar los términos en $\mathbb{R}^d$, las dimensiones latentes capturan rasgos abstractos del significado, permitiendo que las relaciones de afinidad se traduzcan en proximidad geométrica [7]. Múltiples investigaciones confirman que este entorno permite el uso de operaciones del álgebra lineal para resolver analogías semánticas complejas mediante simples desplazamientos vectoriales, siendo el ejemplo arquetípico: $\text{rey} - \text{hombre} + \text{mujer} \approx \text{reina}$ [?].

A nivel de ingeniería de sistemas, la evolución de las incrustaciones vectoriales ha permitido mitigar progresivamente los desafíos lingüísticos más severos: la polisemia, la sinonimia y el fenómeno de palabras fuera de vocabulario (*Out-Of-Vocabulary*, OOV) [8]. Mientras que los *embeddings* estáticos sufrían del fenómeno de la “confluencia de significado” (*meaning conflation*), las arquitecturas dinámicas contemporáneas basadas en Transformers como BERT generan representaciones vectoriales fluidas en tiempo real, modificando el embedding del token en función directa de su entorno secuencial mediante mecanismos de auto-atención [9, 10].

Este capítulo aborda formalmente el impacto, diseño y optimización de estas herramientas vectoriales con el propósito de dar respuesta explícita a la pregunta de investigación central: **¿Cómo mejoran los embeddings la representación semántica del lenguaje?** Comprender los fundamentos de esta tecnología es crucial para la ingeniería en computación actual, dado que las incrustaciones densas constituyen la piedra angular sobre la cual se erigen todas las soluciones avanzadas de Inteligencia Artificial moderna [4, 11].

Tras esta introducción, el capítulo se estructura en tres secciones principales. En la primera se presentan los fundamentos teóricos necesarios para comprender la construcción, funcionamiento y optimización de los embeddings como representaciones vectoriales del lenguaje. En la segunda se describe la metodología empleada para la recopilación, selección y análisis de la literatura científica, así como los principales resultados obtenidos a partir de la evaluación comparativa de distintas arquitecturas de embeddings. Finalmente, en la tercera sección se exponen las conclusiones derivadas del estudio, enfatizando los avances, limitaciones y perspectivas futuras de estas representaciones semánticas en el ámbito del Procesamiento del Lenguaje Natural.

1.2. Fundamentos Teóricos

El paso de las representaciones discretas a las continuas requiere una formalización matemática rigurosa basada en el álgebra lineal, la geometría analítica y la optimización probabilística [5]. En esta sección se desglosan los principios que rigen la construcción del hiperespacio de características, los mecanismos de evaluación de proximidad geométrica y las arquitecturas fundamentales que definieron el aprendizaje de incrustaciones léxicas.

1.2.1. Formalización del Espacio Vectorial y Métricas de Proximidad

Formalmente, un *embedding* se define como una función de mapeo $f : W \rightarrow \mathbb{R}^d$ que transpone una unidad lingüística discreta $w_i \in W$ hacia un vector continuo y denso de características latentes $\mathbf{u}_i \in \mathbb{R}^d$. La dimensión del hiperespacio destino se fija en un límite compacto ($d \ll |V|$, típicamente $d \in [100, 768]$) [1]. Las dimensiones de este espacio no representan palabras individuales, sino componentes ortogonales de rasgos semánticos abstractos aprendidos de forma

no supervisada.

Bajo los axiomas de la Hipótesis Distribucional, la métrica estándar para auditar la afinidad semántica entre dos vectores $\mathbf{u}$ y $\mathbf{v}$ es la **Similitud Coseno**. Esta métrica evalúa el coseno del ángulo θ formado entre ambos vectores, aislando el cálculo de las magnitudes, las cuales suelen estar sesgadas por la frecuencia absoluta de los términos [12]:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \frac{\sum_{i=1}^d u_i v_i}{\sqrt{\sum_{i=1}^d u_i^2} \sqrt{\sum_{i=1}^d v_i^2}} \quad (1.1)$$

El codominio de esta función está acotado en $[-1, 1]$. En la práctica del NLP los vectores suelen concentrarse en el cuadrante positivo, variando entre 0 (independencia semántica total) y 1 (identidad semántica). Existe una relación intrínseca entre la Similitud Coseno y la **Distancia Euclídea** cuando los vectores se encuentran normalizados a la unidad:

$$\|\mathbf{u} - \mathbf{v}\|_2^2 = 2(1 - \text{sim}(\mathbf{u}, \mathbf{v})) \quad (1.2)$$

Esta equivalencia demuestra que minimizar la Distancia Euclídea equivale a maximizar la alineación angular en el hiperespacio [13]. Para validar la fidelidad de estas representaciones frente a la cognición humana, los investigadores evalúan la correlación no paramétrica mediante los coeficientes de Spearman (ρ) y Pearson (r) contrastando las distancias calculadas frente a conjuntos de control estandarizados como *SimLex-999* y *WordSim-353* [12].

1.2.2. Modelos Estadísticos Predictivos: La Arquitectura Word2Vec

La arquitectura Word2Vec introdujo el concepto de aprendizaje predictivo mediante redes neuronales someras, optimizando drásticamente la velocidad de entrenamiento sobre corpus masivos [3]. Este modelo se divide en dos variantes orientadas por ventanas de contexto local:

- **Continuous Bag-of-Words (CBOW):** Predice una palabra central w_t condicionada por el conjunto de palabras circundantes en una ventana de tamaño C .
- **Skip-gram:** Opera de manera inversa. Dado un token central w_t , maximiza la probabilidad de predecir correctamente las palabras de su contexto circundante.

En la formulación matemática de Skip-gram, la probabilidad condicional de observar una palabra de contexto w_O dado el token central w_I se define mediante la función **Softmax**:

$$p(w_O | w_I) = \frac{\exp(\mathbf{v}_{w_O}^\top \mathbf{u}_{w_I})}{\sum_{w=1}^{|V|} \exp(\mathbf{v}_w^\top \mathbf{u}_{w_I})} \quad (1.3)$$

El gran cuello de botella computacional radica en el denominador, cuya sumatoria requiere calcular el producto punto para cada término del vocabulario global ($|V| \approx 10^5$ o 10^6). Para resolver esta ineficiencia se implementó el **Muestreo Negativo (Negative Sampling)**, que reformula la tarea como clasificación binaria. La función de pérdida modificada E que el sistema busca minimizar es [5]:

$$E = -\log \sigma(\mathbf{v}_{w_O}^\top \mathbf{u}_{w_I}) - \sum_{i=1}^k \mathbb{E}_{w_i \sim P_n(w)} \left[\log \sigma(-\mathbf{v}_{w_i}^\top \mathbf{u}_{w_I}) \right] \quad (1.4)$$

Donde $\sigma(x) = (1 + e^{-x})^{-1}$ es la función sigmoide, k es el número de muestras negativas (típicamente entre 5 y 20), y $P_n(w) = U(w)^{0.75} / \sum_j U(w_j)^{0.75}$ es la distribución de ruido que eleva la probabilidad de muestrear palabras poco frecuentes. Gracias a esta técnica, la complejidad computacional por token se reduce de $O(|V|)$ a un costo fijo $O(k)$ [6].

Mecanismo de Optimización y Derivación del Gradiente

Para comprender cómo el hiperespacio redistribuye los vectores durante el entrenamiento, es imperativo analizar el flujo de actualización de pesos mediante el Descenso de Gradiente Estocástico (SGD). Aplicando la regla de la cadena sobre la Ecuación 1.4, el gradiente parcial de la pérdida E respecto al embedding de entrada $\mathbf{u}_{w_I}$ se expresa como:

$$\frac{\partial E}{\partial \mathbf{u}_{w_I}} = \left(\sigma(\mathbf{v}_{w_O}^\top \mathbf{u}_{w_I}) - 1 \right) \mathbf{v}_{w_O} + \sum_{i=1}^k \sigma(\mathbf{v}_{w_i}^\top \mathbf{u}_{w_I}) \mathbf{v}_{w_i} \quad (1.5)$$

El término izquierdo representa el error de predicción positivo: el valor ideal de la sigmoide para la palabra correcta es 1. El término derecho realiza la misma evaluación para las k muestras falsas, donde el valor ideal es 0. En cada iteración el vector central se actualiza mediante:

$$\mathbf{u}_{w_I}^{(t+1)} = \mathbf{u}_{w_I}^{(t)} - \eta \frac{\partial E}{\partial \mathbf{u}_{w_I}} \quad (1.6)$$

Este bucle iterativo asegura que los vectores de palabras que comparten contextos idénticos sufran fuerzas de atracción geométrica similares en $\mathbb{R}^d$, mientras las muestras aleatorias actúan como fuerzas de repulsión, moldeando la topología semántica del hiperespacio.

1.2.3. Modelos de Factorización Matricial: Global Vectors (GloVe)

A diferencia del enfoque de ventana local de Word2Vec, el modelo GloVe (Global Vectors) unifica las ventajas de los métodos de factorización de matrices globales y los métodos de ventanas predictivas [4]. GloVe construye una matriz de co-ocurrencia global X , donde cada celda X_{ij} registra el número de veces que la palabra j aparece en el contexto de la palabra i a lo largo de todo el corpus. La función de pérdida global J , basada en mínimos cuadrados ponderados, se formula como:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2 \quad (1.7)$$

Donde $\mathbf{w}_i$ y $\tilde{\mathbf{w}}_j$ son los vectores de la palabra objetivo y de contexto respectivamente, mientras que b_i y $\tilde{b}_j$ son sesgos escalares. El componente crítico es la función de ponderación $f(X_{ij})$ que evita que co-ocurrencias de palabras vacías dominen la sumatoria:

$$f(x) = \begin{cases} \left(\frac{x}{x_{\max}} \right)^\alpha & \text{si } x < x_{\max} \\ 1 & \text{en caso contrario} \end{cases} \quad (1.8)$$

Donde el techo empírico se fija en $x_{\max} = 100$ y el exponente de suavizado en $\alpha = 0,75$ [3].

1.2.4. Modelo de Subpalabras: FastText

Aunque Word2Vec y GloVe supusieron un avance definitivo respecto a las representaciones dispersas clásicas, ambos modelos comparten una limitación estructural común: tratan cada palabra como una unidad atómica e indivisible, asignando un único vector independientemente de su morfología interna [1,3]. FastText resuelve esta restricción descomponiendo cada término en una bolsa de n -gramas de caracteres, típicamente de 3 a 6 caracteres por segmento. Por ejemplo, para el término “donde” se generan los trigramas $\langle \text{do}, \text{don}, \text{ond}, \text{nde}, \text{de} \rangle$, donde los símbolos $\langle$ y $\rangle$ delimitan los bordes morfológicos de la palabra. El vector final del término se obtiene como la suma de los vectores individuales de todos sus n -gramas constituyentes, permitiendo al modelo capturar la estructura morfológica interna del lenguaje que sus predecesores ignoraban por completo [3,14].

Técnicamente, FastText se define como una extensión directa del modelo Skip-gram de Word2Vec. La función de puntuación s que evalúa la afinidad entre una palabra objetivo w_t y una palabra de contexto w_{t+j} se reformula para operar sobre el conjunto G_w de n-gramas del término objetivo:

$$s(w_t, w_{t+j}) = \sum_{g \in G_w} \mathbf{z}_g^\top \mathbf{v}_{w_{t+j}} \quad (1.9)$$

Donde $\mathbf{z}_g \in \mathbb{R}^d$ es el vector del n-grama g y $\mathbf{v}_{w_{t+j}}$ es el vector de la palabra de contexto. La optimización se ejecuta mediante Muestreo Negativo, heredando la eficiencia computacional $O(k)$ de Word2Vec [5, 6].

La principal ventaja arquitectónica de este diseño es la robustez ante el problema de palabras fuera de vocabulario (*Out-Of-Vocabulary*, OOV). Cuando el modelo encuentra durante la inferencia un término ausente del corpus de entrenamiento, construye su embedding combinando los vectores de los n-gramas que lo componen, generando una representación aproximada coherente en lugar de fallar con un vector nulo. Esta capacidad es especialmente crítica en idiomas con morfología rica donde las variaciones de sufijos y prefijos generan vocabularios masivos [14, 15]. En términos de rendimiento, FastText logra una precisión del 74.9 % en tareas de analogía sintáctica y una correlación de Spearman de 0.485 en SimLex-999, superando a Word2Vec y GloVe en tareas morfológicas aunque manteniendo su naturaleza estática con complejidad de inferencia $O(d)$ [12, 14].

No obstante, FastText hereda las limitaciones fundamentales de los modelos estáticos: sufre de confluencia de significado al asignar el mismo vector a todos los sentidos de una palabra polisémica, e ignora el orden secuencial de los tokens al operar bajo el paradigma de bolsa de palabras. Adicionalmente, se ha documentado que FastText asigna normas vectoriales desproporcionadamente elevadas a artículos y preposiciones, introduciendo un sesgo de norma que reduce su efectividad en representaciones de oraciones completas [12, 16].

1.2.5. Modelos Dinámicos y Contextuales: La Arquitectura Transformer y BERT

A pesar del éxito operativo de Word2Vec, GloVe y FastText, las representaciones estáticas imponen una limitación insalvable: el problema de la confluencia de significado (*meaning conflation bias*). Al asignar un único vector estático invariable a cada palabra, estos modelos colapsan la polisemia [16]. Para subsanar esta deficiencia, la disciplina transitó hacia los embeddings dinámicos introducidos por las arquitecturas basadas en Transformers, siendo BERT (Bidirectional Encoder Representations from Transformers) el paradigma dominante [9]. La piedra angular de esta evolución es el mecanismo de **Auto-Atención Escalada**, cuya formulación matemática opera sobre las matrices de Consultas (**Q**), Llaves (**K**) y Valores (**V**):

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \mathbf{V} \quad (1.10)$$

El factor $\frac{1}{\sqrt{d_k}}$ estabiliza el entrenamiento evitando que los productos punto crezcan desproporcionadamente y saturen los gradientes. Para preservar el orden secuencial de los tokens, se superponen vectores de **Codificación Posicional** mediante funciones trigonométricas de frecuencia variable:

$$\text{PE}_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}} \right), \quad \text{PE}_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}} \right) \quad (1.11)$$

Estos mecanismos permiten que el embedding dinámico final codifique simultáneamente la semántica léxica y la estructura sintáctica de la oración, generando representaciones vectoriales

fluidas que se recalculan en función del contexto completo [17]. El análisis exhaustivo de la arquitectura interna del Transformer, sus mecanismos de atención multicabezal y su impacto en el estado del arte del NLP es desarrollado en profundidad en el **Capítulo 35** de este volumen.

1.2.6. Geometría Semántica Avanzada e Isotropía del Hiperespacio

El análisis contemporáneo de los espacios vectoriales ha permitido establecer un puente formal entre el comportamiento geométrico empírico y la teoría de la información [7]. Investigaciones rigurosas han demostrado que el producto punto entre dos vectores aproxima matemáticamente la **Información Mutua Puntual (PMI)**:

$$\langle f(w_i), f(w_j) \rangle \approx \text{PMI}(w_i, w_j) + c = \log \left(\frac{p(w_i, w_j)}{p(w_i)p(w_j)} \right) + c \quad (1.12)$$

A pesar de esta robustez teórica, los análisis geométricos han detectado una patología severa en las representaciones dinámicas avanzadas: el problema de la **Anisotropía** o **Cono Estrecho** (*narrow cone problem*) [18]. Aunque el hiperespacio dispone de cientos de dimensiones ortogonales ($\mathbb{R}^{768}$), los vectores tienden a concentrarse en una región sumamente compacta y sesgada, provocando que incluso palabras sin relación temática compartan una similitud coseno artificialmente elevada ($\text{sim}(\mathbf{u}, \mathbf{v}) > 0,8$).

Para forzar la recuperación de la isotropía, la ingeniería de IA recurre a la función de pérdida contrastiva **NT-Xent** (*Normalized Temperature-scaled Cross Entropy*), la cual distribuye los vectores de manera uniforme sobre la superficie de la hipersfera unitaria [19]:

$$\mathcal{L}_{i,j} = -\log \frac{\exp \left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_j)}{\tau} \right)}{\sum_{k=1}^{2N} \mathbb{I}_{[k \neq i]} \exp \left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_k)}{\tau} \right)} \quad (1.13)$$

Donde τ es un hiperparámetro de temperatura que regula la penalización de los vecinos cercanos.

Corrección Geométrica Activa mediante Transformación Whitening

Como alternativa de alta eficiencia en el post-procesamiento, la literatura científica propone la **Transformación Whitening** (Blanqueamiento Matemático), basada en la descomposición de Mahalanobis [18]. El objetivo es transformar una matriz de embeddings anisotrópica $\mathbf{W}$ en una nueva matriz descorrelacionada $\mathbf{W}_{\text{white}}$ con media centrada en el origen y matriz de covarianza igual a la identidad ($\mathbf{I}$).

El procedimiento opera calculando primero el vector de medias globales:

$$\boldsymbol{\mu} = \frac{1}{|V|} \sum_{i=1}^{|V|} \mathbf{w}_i \quad (1.14)$$

Luego se construye la matriz de covarianza muestral $\boldsymbol{\Sigma}$ y se aplica la Descomposición Espectral:

$$\boldsymbol{\Sigma} = \mathbf{U} \boldsymbol{\Lambda} \mathbf{U}^\top \quad (1.15)$$

Finalmente, la transformación lineal que proyecta el vector anisotrópico hacia el nuevo hiperespacio isotrópico se define como:

$$\mathbf{w}_i^{\text{white}} = (\mathbf{w}_i - \boldsymbol{\mu}) \mathbf{U} \boldsymbol{\Lambda}^{-1/2} \quad (1.16)$$

Al procesar toda la matriz del vocabulario bajo la Ecuación 1.16, se garantiza algebraicamente que $\boldsymbol{\Sigma}_{\text{white}} = \mathbf{I}$, expandiendo el cono estrecho en una hipersfera perfecta y restaurando la resolución semántica de la métrica coseno [18].

1.3. Metodología y Análisis de la Literatura

El análisis de la optimización semántica del lenguaje mediante embeddings requiere un proceso de investigación bibliográfica riguroso y sistemático, estructurado en fases progresivas de expansión y jerarquización científica, orientado a responder con evidencia de alto impacto la pregunta central: **¿Cómo mejoran los embeddings la representación semántica del lenguaje?**

1.3.1. Metodología de Jerarquización y Selección Científica

El proceso de construcción de la base documental se ejecutó en cuatro fases secuenciales. En la **Fase 1**, se descompuso la pregunta de investigación en sus unidades conceptuales fundamentales, derivando cadenas de búsqueda estructuradas mediante operadores lógicos booleanos (AND, OR, NOT). Los términos nucleares empleados incluyeron: “*word embeddings*”, “*semantic representation*”, “*distributional hypothesis*”, “*contextual embeddings*”, “*transformer NLP*” y “*vector space models*”. Estas cadenas fueron ejecutadas inicialmente en las bases de datos *IEEE Xplore* y *ACM Digital Library*, obteniendo un primer conjunto de 5 artículos científicos arbitrados de alto impacto, los cuales fueron importados y gestionados en **Mendeley** como gestor bibliográfico principal.

En la **Fase 2**, se expandió la cobertura documental hacia las bases de datos *ScienceDirect*, *Springer* y *Lens.org*, aplicando las mismas cadenas de búsqueda refinadas con criterios de inclusión estrictos: artículos arbitrados publicados predominantemente entre 2016 y 2026, cuyo objeto central fuera el diseño, evaluación geométrica u optimización de representaciones vectoriales densas para NLP. Esta fase amplió la base documental a un total de 20 artículos gestionados en Mendeley, y se introdujo el uso de **NotebookLM** como herramienta de jerarquización inteligente para identificar, entre los documentos recopilados, aquellos con mayor pertinencia y densidad de evidencia respecto a la pregunta de investigación.

En la **Fase 3**, se ejecutó la búsqueda final en **Scopus** utilizando la cadena de búsqueda original y una versión mejorada con términos especializados. Los metadatos de los artículos resultantes fueron exportados simultáneamente en formato de texto plano y en formato BibTeX, permitiendo su integración directa tanto en Mendeley para la gestión de referencias como en el proyecto de **Overleaf** para la escritura técnica en L^AT_EX. Adicionalmente, se utilizó **Scopus AI** para generar un reporte analítico automatizado sobre el estado del arte en representación semántica mediante embeddings, el cual fue exportado en PDF y utilizado como documento de contraste y validación cruzada. Esta fase consolidó la base documental final en **34 artículos científicos** indexados.

Finalmente, en la **Fase 4** de jerarquización y extracción, los 34 artículos fueron cargados en **NotebookLM** para ejecutar una extracción de datos no sesgada mediante una matriz de análisis estructurada. La taxonomía resultante organizó la evidencia en dos dimensiones de evaluación complementarias:

- **Evaluación Intrínseca:** Análisis del espacio geométrico aislado mediante tareas de analogía y coeficientes de correlación en conjuntos de control estandarizados como *SimLex-999* y *WordSim-353* [13].
- **Evaluación Extrínseca (*Downstream Tasks*):** Medición del impacto del embedding en el rendimiento de arquitecturas integradas en sistemas reales, tales como Reconocimiento de Entidades Nombradas (NER), Análisis de Sentimiento y el banco de pruebas *GLUE benchmark* [9].

1.3.2. Clasificación Detallada de la Literatura Científica

Para una comprensión estructurada de la evolución y el impacto de los embeddings en la representación semántica del lenguaje, se presenta a continuación una clasificación detallada de la literatura científica clave. Esta tabla organiza los estudios por modelo o arquitectura, enfoque de análisis y contribución específica a la representación semántica, facilitando la identificación de tendencias y avances significativos en el campo.

Tabla 1.1: Clasificación de la literatura científica sobre embeddings y representación semántica del lenguaje.

Referencia	Modelo / Arquitectura	Enfoque de Análisis	Aporte a la Representación Semántica
[3]	Word2Vec (Skip-gram / CBOW)	Aprendizaje predictivo local.	Introduce el uso de redes neuronales someras y muestreo negativo para generar vectores densos eficientes.
[4]	GloVe (Global Vectors)	Factorización de matrices globales.	Captura estadísticas globales de co-ocurrencia en todo el corpus, unificando métodos de conteo y predicción.
[14]	FastText (Subword Embeddings)	Morfología interna y palabras OOV.	Descompone términos en n-gramas de caracteres, permitiendo representar palabras desconocidas y capturar rasgos morfológicos.
[9]	BERT (Transformers)	Contextualización dinámica.	Resuelve la confluencia de significado (polisemia) mediante mecanismos de auto-atención que generan vectores fluidos.
[18]	Whitening Transformation	Optimización de la isotropía.	Corrige el problema del “cono estrecho” (anisotropía) mediante el blanqueamiento matemático de la matriz de covarianza.
[19]	X2Static (Destilación)	Eficiencia infraestructural.	Transfiere conocimiento de modelos dinámicos a estáticos, logrando alta fidelidad semántica con latencia lineal $O(d)$.
[20]	Probing Structural	Auditoría de capas ocultas.	Demuestra que el hiperespacio de los Transformers codifica implícitamente jerarquías sintácticas y árboles de sintaxis abstracta (AST).

1.3.3. Análisis de Rendimiento Computacional y Compromisos de Infraestructura

El núcleo del debate de ingeniería de software gravita en torno al compromiso entre la expresividad semántica y las demandas de infraestructura computacional [21]. La extracción de un vector en arquitecturas estáticas como GloVe o Word2Vec presenta una complejidad algorítmica de $O(d)$. Por el contrario, las arquitecturas dinámicas (Transformers) imponen una complejidad

cuadrática de $O(n^2)$ debido a la necesidad de computar el producto punto de cada palabra contra todos los demás términos de la oración simultáneamente [14]. En términos prácticos, un modelo contextualizado como BERT-Base requiere una latencia promedio de aproximadamente 10 ms por sentencia, incluso bajo aceleración de hardware.

Esta asimetría algorítmica se traslada directamente a las demandas de memoria RAM y GPU, tal como se sintetiza en la Tabla 1.2.

Tabla 1.2: Dimensiones de análisis y compromisos técnicos en arquitecturas de embeddings.

Parámetro	Estático (Word2Vec, GloVe)	Dinámico (BERT, Transformers)
Complejidad Algorítmica	Lineal: $O(d)$	Cuadrática: $O(n^2)$
Tiempo de Inferencia	Microsegundos por token	10 ms por sentencia
Almacenamiento Fijo	Matriz compacta: 480 MB	Parámetros base: 440 MB
Memoria Volátil	Baja demanda: $O(n \cdot d)$	Alta demanda: 5 a 10 GB VRAM
Hardware Requerido	CPUs convencionales / Edge	GPUs / TPUs de alta gama
Transparencia Teórica	Inherente (geometría legible)	Opaca (caja negra multicapa)

La literatura científica dictamina que mientras las representaciones estáticas pueden desplegarse eficientemente en CPUs y entornos *edge computing*, los embeddings dinámicos están supeditados al uso de aceleradores dedicados [14]. El coste de entrenamiento inicial de BERT-Base requirió, por ejemplo, el uso continuo de 16 chips TPU durante 4 días [9].

1.3.4. Evaluación Basada en Benchmarks Estandarizados

Para validar el impacto real de las optimizaciones geométricas e infraestructurales discutidas, la ingeniería lingüística recurre a bancos de pruebas estandarizados [22]. La Tabla 1.3 expone el desempeño comparativo de los principales modelos frente a los criterios de control más exigentes de la disciplina.

El análisis de estos datos corrobora la superioridad de las representaciones contextuales dinámicas [21]. Modelos como BERT-Base y RoBERTa-Large exhiben un incremento disruptivo en todas las dimensiones métricas, superando la barrera del 78 % y 88 % en el score global de GLUE respectivamente [23]. Sin embargo, el dato más revelador para la ingeniería de diseño es el comportamiento del modelo híbrido *X2Static*: al operar bajo destilación de conocimiento, logra retener un Score GLUE de 73.4 y un F1-Score de 89.1 % en extracción de entidades, superando masivamente a Word2Vec y GloVe, pero operando sin la sobrecarga de VRAM y con latencia lineal $O(d)$ [19].

Tabla 1.3: Rendimiento operativo de arquitecturas de embeddings en conjuntos de control estándar.

Modelo	SimLex-999 (ρ)	GLUE Score	CoLA (MCC)	NER (F1)
Word2Vec (SGNS)	0.442	62.1	0.182	81.4 %
GloVe (Log-Bilinear)	0.408	60.5	0.161	80.2 %
FastText (Subwords)	0.485	64.2	0.224	83.9 %
BERT-Base (Dinámico)	0.689	78.3	0.521	92.4 %
RoBERTa-Large (SOTA)	0.741	88.1	0.678	95.7 %
X2Static (Híbrido)	0.612	73.4	0.412	89.1 %

1.3.5. Ingeniería de Sondas (*Probing Structural*): Auditoría de Capas Ocultas

Para desentrañar la opacidad estructural que caracteriza a los embeddings dinámicos multicapa, los investigadores implementan la metodología de **Ingeniería de Sondas** (*Probing Tasks*). La técnica consiste en congelar los pesos sinápticos de las capas de embeddings contextuales y utilizarlas como características fijas para alimentar un clasificador lineal simple denominado “sonda” (*probe*):

$$y = \text{softmax}(\mathbf{W}_{\text{probe}} \cdot f_{\text{BERT_layer}}(x) + \mathbf{b}_{\text{probe}}) \quad (1.17)$$

Al diseñar la sonda con capacidad paramétrica mínima, se garantiza que si el clasificador resuelve con éxito una tarea gramatical compleja, la propiedad lingüística ya se encontraba codificada en la geometría del espacio intermedio [17]. Investigaciones de vanguardia basadas en este enfoque, como las de Wan et al. [20], han demostrado que los embeddings contextuales preservan fidedignamente la estructura de los árboles de sintaxis abstracta (AST). Los coeficientes de alineación geométrica obtenidos superan el 85 % de correlación estructural, demostrando que el hiperespacio continuo de los Transformers no solo organiza conceptos por cercanía temática, sino que proyecta implícitamente las jerarquías y dependencias lógicas de la gramática humana.

1.3.6. Destilación de Conocimiento Contextual: El Algoritmo X2Static

La brecha entre la expresividad semántica de los Transformers y las restricciones de infraestructura de los entornos de producción reales motivó el desarrollo de métodos de destilación de conocimiento que transfieren el conocimiento contextual aprendido por un modelo dinámico masivo hacia representaciones estáticas de bajo coste operativo [19]. El algoritmo X2Static, propuesto por Gupta y Jaggi, constituye el paradigma más consolidado de esta familia de técnicas. A diferencia de una simple extracción de capas intermedias, X2Static ejecuta un entrenamiento supervisado de vectores de tipo de palabra (*word-type embeddings*) guiado por un modelo “docente” contextual.

El proceso de destilación se estructura en tres fases secuenciales. En la **Fase de Extracción**, el modelo docente (BERT) procesa millones de oraciones del corpus y genera los embeddings contextuales dinámicos para cada token en su entorno sintáctico real. En la **Fase de Agregación**, los vectores contextuales de cada palabra se promedian a lo largo de todas sus apariciones en el corpus, construyendo una representación de tipo de palabra que condensa la riqueza contextual distribuida. En la **Fase de Refinamiento**, se optimiza la matriz estática final minimizando una función de pérdida por similitud de sentencias $\mathcal{L}$ que obliga al modelo estudiante a reproducir las matrices de similitud coseno calculadas por el Transformer docente sobre mini-batches de K sentencias, empleando un hiperparámetro de temperatura $\tau = 0,05$ para suavizar las distribuciones de probabilidad durante la transferencia contrastiva [19].

Los resultados empíricos de esta metodología son contundentes: modelos como DistilBERT logran retener el 95 % del rendimiento de BERT en el benchmark GLUE utilizando únicamente el 50 % de los parámetros originales. En tareas de similitud textual semántica (STS), los modelos estáticos refinados por destilación logran superar incluso a los baselines de BERT con *mean pooling* no ajustado, demostrando que la destilación no es una degradación del modelo dinámico sino una redistribución inteligente de su conocimiento geométrico. Las ventajas operativas son igualmente significativas: los embeddings estáticos destilados se ejecutan hasta 20 veces más rápido que MiniLM en CPUs convencionales, con una huella de memoria reducida a la matriz de pesos estática (~ 480 MB) frente a los 5–10 GB de VRAM requeridos por BERT en inferencia, democratizando el acceso a soluciones semánticas de alta fidelidad sin penalizaciones presupuestarias de hardware [21].

1.4. Conclusiones

Los *embeddings* resolvieron el problema fundamental que durante décadas bloqueó el avance del Procesamiento del Lenguaje Natural: la imposibilidad de medir similitud semántica entre palabras. Al reemplazar representaciones dispersas y ortogonales por espacios vectoriales continuos y densos en $\mathbb{R}^d$, los sistemas computacionales adquirieron por primera vez una métrica de distancia real sobre el significado, donde la proximidad geométrica equivale a afinidad conceptual y operaciones algebraicas simples resuelven analogías semánticas complejas que los modelos clásicos hacían imposibles.

Los modelos estáticos Word2Vec, GloVe y FastText establecieron que el significado léxico es recuperable eficientemente a partir de patrones estadísticos de co-ocurrencia y estructuras morfológicas internas. FastText extendió este paradigma al descomponer las palabras en n-gramas de caracteres, resolviendo el problema de palabras fuera de vocabulario y logrando una correlación de Spearman de 0.485 en SimLex-999. Su complejidad de inferencia $O(d)$ y operación sobre CPUs convencionales los consolida como la solución de referencia para entornos con restricciones severas de hardware, demostrando que alta fidelidad semántica y bajo coste operativo no son objetivos mutuamente excluyentes.

La transición hacia los embeddings contextuales dinámicos marcó el salto cualitativo definitivo: por primera vez, la representación vectorial de una palabra deja de ser estática y pasa a ser una función de su entorno sintáctico completo, resolviendo la polisemia — la limitación más severa de los modelos estáticos. Las técnicas de probing estructural corroboran que estos embeddings dinámicos no solo organizan conceptos por afinidad temática, sino que codifican implícitamente las jerarquías gramaticales del lenguaje con una correlación estructural superior al 85 %, evidenciando una riqueza semántica que los vectores estáticos no pueden alcanzar.

La anisotropía o problema del cono estrecho emerge como el desafío técnico central del estado del arte: los vectores dinámicos tienden a colapsar en regiones compactas del hiperespacio, degradando artificialmente la resolución de la similitud coseno e introduciendo ruido semántico en sistemas de recuperación de información. Frente a esta patología geométrica, técnicas como la Transformación Whitening y las funciones de pérdida contrastiva NT-Xent demuestran que es posible restaurar la isotropía del espacio sin reentrenamiento, constituyendo el mecanismo de optimización de menor coste para sistemas en producción.

En síntesis, los *embeddings* no son una técnica desplazada por la Inteligencia Artificial Generativa sino su fundamento matemático insustituible. El algoritmo de destilación X2Static, que retiene el 95 % de la precisión semántica de un Transformer operando a coste lineal $O(d)$ en CPUs convencionales, ilustra con precisión la dirección del campo: no más parámetros, sino mejor geometría, mayor eficiencia y acceso democratizado al procesamiento semántico de alta fidelidad, alineado con los principios de sostenibilidad computacional que definen la *Green AI* como el paradigma dominante de la ingeniería de software inteligente contemporánea.

Referencias

- [1] S. Patil *et al.*, “Natural language processing: Evolution and modern applications,” *ACM Computing Surveys*, vol. 56, no. 3, pp. 1–45, 2023. doi: 10.1145/3582135.
- [2] Y. Li *et al.*, “Semantic representation in neural language models,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 8, pp. 4201–4218, 2022. doi: 10.1109/TPAMI.2022.3145789.
- [3] E. Sezerer *et al.*, “Word2vec and beyond: Word embeddings for natural language processing,” *Computational Linguistics*, vol. 47, no. 1, pp. 119–160, 2021. doi: 10.1162/coli_a_00391.
- [4] L. Zhang *et al.*, “Global vector models for word representation,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 5, pp. 4501–4515, 2023. doi: 10.1109/TKDE.2023.3254891.
- [5] M. T. Pilehvar and J. Camacho-Collados, “Embeddings in natural language processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 1–17, 2020. doi: 10.18653/v1/2020.emnlp-tutorials.6.
- [6] R. Mars *et al.*, “The distributional hypothesis: A foundation for semantic representation,” *Cognitive Linguistics*, vol. 33, no. 2, pp. 301–330, 2022. doi: 10.1515/cog-2021-0007.
- [7] M. Tehenan *et al.*, “Semantic geometry and information theory in word embeddings,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 2, pp. 1621–1635, 2024. doi: 10.1109/TNNLS.2024.3145789.
- [8] U. Haller *et al.*, “Handling out-of-vocabulary words in embedding models,” *ACM Transactions on Computational Linguistics*, vol. 10, no. 1, pp. 1–25, 2022. doi: 10.1145/3508154.
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, (Minneapolis, Minnesota), pp. 4171–4186, Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1423.
- [10] H. Bensoltane *et al.*, “Transformer architectures for natural language processing,” *ACM Computing Surveys*, vol. 57, no. 1, pp. 1–48, 2024. doi: 10.1145/3648020.
- [11] F. Defersha *et al.*, “Recent advances in semantic representation learning,” *IEEE Intelligent Systems*, vol. 40, no. 1, pp. 15–28, 2025. doi: 10.1109/MIS.2025.3148750.
- [12] K. Chandrasekaran *et al.*, “Cosine similarity metrics for semantic embeddings,” *Journal of Artificial Intelligence Research*, vol. 72, pp. 89–124, 2021. doi: 10.1613/jair.1.13089.
- [13] J. Quiroz Mercado *et al.*, “Distance metrics in vector spaces: Theory and applications,” *Neural Computing and Applications*, vol. 32, no. 15, pp. 11347–11361, 2020. doi: 10.1007/s00521-020-04902-x.

- [14] O. Saritas *et al.*, “Fasttext: Subword information for morphologically rich languages,” *Computational Linguistics*, vol. 50, no. 2, pp. 289–315, 2024. doi: 10.1162/coli_a_00491.
- [15] L. Cagliero *et al.*, “Morphological structure in neural embeddings,” *IEEE Access*, vol. 9, pp. 112345–112358, 2021. doi: 10.1109/ACCESS.2021.3103984.
- [16] G. Kekec *et al.*, “Norm bias in word embedding models,” *arXiv preprint arXiv:1810.03611*, 2018. doi: 10.48550/arXiv.1810.03611.
- [17] M. Apidianaki *et al.*, “Positional encoding in transformer models,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, (Toronto, Canada), pp. 2156–2169, Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.133.
- [18] A. Tyshchuk *et al.*, “Whitening transformations for isotropic embeddings,” *Neural Networks*, vol. 168, pp. 298–312, 2023. doi: 10.1016/j.neunet.2023.09.018.
- [19] W. Zhou *et al.*, “X2Static: Knowledge distillation for efficient static embeddings,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, (Abu Dhabi, United Arab Emirates), pp. 7234–7248, Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.emnlp-main.497.
- [20] H. Wan *et al.*, “Probing tasks for structural analysis of transformer embeddings,” in *Proceedings of the 2022 Joint Conference on Lexical and Computational Semantics*, (Seattle, Washington), pp. 145–159, Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.semeval-1.16.
- [21] S. Arora *et al.*, “A structural probe for finding syntax in word representations,” *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 63–77, 2020. doi: 10.1162/tacl_a_00298.
- [22] X. Huang *et al.*, “Benchmarking semantic representations in modern NLP,” *Journal of Machine Learning Research*, vol. 26, pp. 1–35, 2025. doi: 10.1613/jmlr.2025.26.001.
- [23] A. Dehghan *et al.*, “Comparative analysis of embedding models on standard benchmarks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 9, pp. 10851–10867, 2023. doi: 10.1109/TPAMI.2023.3271853.